```python
!pip install -q transformers datasets evaluate peft
import torch
import numpy as np
import random
from datasets import load_dataset
from transformers import (
DistilBertTokenizerFast,
DistilBertForSequenceClassification,
Trainer,
TrainingArguments
)
import evaluate
from peft import LoraConfig, get_peft_model, TaskType
SEED = 42
random.seed(SEED)
np.random.seed(SEED)
torch.manual_seed(SEED)
torch.cuda.manual_seed_all(SEED)
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Using device:", device)

dataset = load_dataset("imdb")
train_dataset = dataset["train"].shuffle(seed=SEED).select(range(10000))
test_dataset = dataset["test"].shuffle(seed=SEED).select(range(2000))
tokenizer = DistilBertTokenizerFast.from_pretrained("distilbert-base-
         uncased")
def tokenize(batch):
    return tokenizer(
        batch["text"],
        truncation=True,
        padding="max_length",
        max_length=256
    )
train_dataset = train_dataset.map(tokenize, batched=True)
test_dataset = test_dataset.map(tokenize, batched=True)
train_dataset.set_format("torch", columns=["input_ids", "attention_mask",
         "label"])
test_dataset.set_format("torch", columns=["input_ids", "attention_mask",
         "label"])
accuracy_metric = evaluate.load("accuracy")
f1_metric = evaluate.load("f1")
def compute_metrics(eval_pred):
    logits, labels = eval_pred
    preds = np.argmax(logits, axis=-1)
    acc = accuracy_metric.compute(predictions=preds, references=labels)
    f1 = f1_metric.compute(predictions=preds, references=labels)
    return {
        "accuracy": acc["accuracy"],
```

```
        "f1": f1["f1"]
    }
baseline_model = DistilBertForSequenceClassification.from_pretrained(
    "distilbert-base-uncased",
    num_labels=2
)
# Freeze transformer layers
for param in baseline_model.distilbert.parameters():
    param.requires_grad = False
baseline_model.to(device)
training_args = TrainingArguments(
    output_dir="./results",
    per_device_train_batch_size=16,
    per_device_eval_batch_size=16,
    num_train_epochs=2,
    eval_strategy="epoch",
    logging_strategy="epoch",
    save_strategy="no",
    learning_rate=2e-5,
    weight_decay=0.01,
    report_to="none",
)
baseline_trainer = Trainer(
    model=baseline_model,
    args=training_args,
    train_dataset=train_dataset,
    eval_dataset=test_dataset,
    compute_metrics=compute_metrics,
)
baseline_trainer.train()
baseline_results = baseline_trainer.evaluate()
print("Baseline Results:", baseline_results)

full_model = DistilBertForSequenceClassification.from_pretrained(
    "distilbert-base-uncased",
    num_labels=2
)
full_model.to(device)
full_trainer = Trainer(
    model=full_model,
    args=training_args,
    train_dataset=train_dataset,
    eval_dataset=test_dataset,
    compute_metrics=compute_metrics,
)
full_trainer.train()
full_results = full_trainer.evaluate()
print("Full Fine-Tuning Results:", full_results)
lora_model = DistilBertForSequenceClassification.from_pretrained(
    "distilbert-base-uncased",
    num_labels=2
```

```
)
lora_config = LoraConfig(
    task_type=TaskType.SEQ_CLS,
    r=8,
    lora_alpha=16,
    lora_dropout=0.1,
    target_modules=["q_lin", "v_lin"]
)
lora_model = get_peft_model(lora_model, lora_config)
lora_model.print_trainable_parameters()
lora_model.to(device)
lora_trainer = Trainer(
    model=lora_model,
    args=training_args,
    train_dataset=train_dataset,
    eval_dataset=test_dataset,
    compute_metrics=compute_metrics,
)
lora_trainer.train()
lora_results = lora_trainer.evaluate()
print("LoRA Results:", lora_results)
print("\n===== FINAL RESULTS =====")
print("Baseline:", baseline_results)
print("Full Fine-Tuning:", full_results)
print("LoRA (PEFT):", lora_results)
raw_test_dataset = load_dataset("imdb")
         ["test"].shuffle(seed=42).select(range(2000))
def get_predictions(trainer, tokenized_dataset, raw_dataset, n=5):
    outputs = trainer.predict(tokenized_dataset)
    preds = np.argmax(outputs.predictions, axis=-1)
    labels = outputs.label_ids
    for i in range(n):
        print("TEXT:", raw_dataset[i]["text"][:300])
        print("TRUE:", labels[i], "PRED:", preds[i])
        print("-"*70)
print("Baseline examples:")
get_predictions(baseline_trainer, test_dataset, raw_test_dataset)
print("LoRA examples:")
get_predictions(lora_trainer, test_dataset, raw_test_dataset)
print("Full FT examples:")
get_predictions(full_trainer, test_dataset, raw_test_dataset)
import matplotlib.pyplot as plt
import numpy as np
# ===== Results (from your FINAL RESULTS) =====
models = ["Baseline\n(Frozen)", "LoRA\n(PEFT)", "Full FT\n(FFT)"]
accuracy = np.array([0.8065, 0.8575, 0.9085])
f1 = np.array([0.8094534711964549, 0.8579970104633782, 0.9081786251881585])
loss = np.array([0.5333009362220764, 0.32603591680526733,
         0.25742846727371216])
# NOTE: replace these with the EXACT trainable param counts if you printed
# Baseline (classifier head only) ~0.8M is typical; Full FT ~66M for
# DistilBERT.
```

```python
trainable_m = np.array([0.8, 2.0, 66.0])  # Millions
# ------------------------
# Fig 1: Accuracy & F1 comparison
# ------------------------
x = np.arange(len(models))
width = 0.35
plt.figure(figsize=(8,5))
plt.bar(x - width/2, accuracy, width, label="Accuracy")
plt.bar(x + width/2, f1, width, label="F1")
plt.xticks(x, models)
plt.ylim(0.75, 0.95)
plt.ylabel("Score")
plt.title("IMDB Performance Comparison (Accuracy & F1)")
plt.legend()
plt.tight_layout()
plt.savefig("fig1_accuracy_f1.png", dpi=300)
plt.close()
# ------------------------
# Fig 2: Evaluation loss
# ------------------------
plt.figure(figsize=(7,5))
plt.bar(models, loss)
plt.ylabel("Eval Loss")
plt.title("Evaluation Loss Across Training Regimes")
plt.tight_layout()
plt.savefig("fig2_eval_loss.png", dpi=300)
plt.close()
# ------------------------
# Fig 3: Trainable parameter efficiency (log scale)
# ------------------------
plt.figure(figsize=(7,5))
plt.bar(models, trainable_m)
plt.yscale("log")
plt.ylabel("Trainable Parameters (Millions, log scale)")
plt.title("Parameter Efficiency: Trainable Parameters by Method")
plt.tight_layout()
plt.savefig("fig3_trainable_params_log.png", dpi=300)
plt.close()
# ------------------------
# Fig 4: Efficiency-performance trade-off (log x)
# ------------------------
plt.figure(figsize=(7,5))
plt.scatter(trainable_m, accuracy)
for i, m in enumerate(models):
    plt.annotate(m.replace("\n", " "), (trainable_m[i], accuracy[i]),
                 textcoords="offset points", xytext=(6,6))
plt.xscale("log")
plt.xlabel("Trainable Parameters (Millions, log scale)")
plt.ylabel("Accuracy")
plt.title("Efficiency–Performance Trade-off")
plt.tight_layout()
```

```
plt.savefig("fig4_efficiency_tradeoff.png", dpi=300)
plt.close()
print("Saved figures:")
print("fig1_accuracy_f1.png")
print("fig2_eval_loss.png")
print("fig3_trainable_params_log.png")
print("fig4_efficiency_tradeoff.png")
```

```
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 0.0/84.1 kB ? eta -:--:--
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 84.1/84.1 kB 4.1 MB/s eta 0:00:00

/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94:
UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings
tab (https://huggingface.co/settings/tokens), set it as secret in your Google
Colab and restart your session.
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access
public models or datasets.
  warnings.warn(
```

{"model_id":"02f699e9a00c4c539b09fcf26bdffbee","version_major":2,"version_minor":0}

```
Warning: You are sending unauthenticated requests to the HF Hub. Please set a
HF_TOKEN to enable higher rate limits and faster downloads.
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated
requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits
and faster downloads.
```

{"model_id":"d022397deddc49ae9949dac7165b665e","version_major":2,"version_minor":0}

{"model_id":"039714ff35384be695fd64415477eb77","version_major":2,"version_minor":0}

{"model_id":"ca543d06ca9c467492d9f7845569f314","version_major":2,"version_minor":0}

{"model_id":"53902e5b6ebf4f8ba41637fed89981ee","version_major":2,"version_minor":0}

{"model_id":"c280afaac1bd41f883388f4ba0acdf52","version_major":2,"version_minor":0}

{"model_id":"e64ebb65d13e426a988a238e4431d479","version_major":2,"version_minor":0}

{"model_id":"cdf35499d3a045acb8908b99891fd5e3","version_major":2,"version_minor":0}

{"model_id":"0d405757c24c489ca85c9b9a19134e3c","version_major":2,"version_minor":0}

{"model_id":"a965218b91454cb6b15b70ab82f424ed","version_major":2,"version_minor":0}

{"model_id":"0b45a84db2dc45239c3e3d1860e07a7b","version_major":2,"version_minor":0}

{"model_id":"f6845f6a3965487899df5fa04ce7c032","version_major":2,"version_minor":0}

{"model_id":"09acbf7272844fe5b66ad4a3a8d53bb5","version_major":2,"version_minor":0}

{"model_id":"38785b20ba35470e9e6b9617d1dd0ae8","version_major":2,"version_minor":0}

{"model_id":"281beb4cc3044a20a6b83d6f8be0353f","version_major":2,"version_minor":0}

{"model_id":"b047698467974e178d53a7b0aad482f7","version_major":2,"version_minor":0}

{"model_id":"3969f1d2f4db452a862cf039b9ce87fa","version_major":2,"version_minor":0}

```
DistilBertForSequenceClassification LOAD REPORT from: distilbert-base-uncased
Key                      | Status     |
-------------------------+------------+-
vocab_transform.weight   | UNEXPECTED |
vocab_transform.bias     | UNEXPECTED |
vocab_layer_norm.weight  | UNEXPECTED |
vocab_projector.bias     | UNEXPECTED |
vocab_layer_norm.bias    | UNEXPECTED |
pre_classifier.bias      | MISSING    |
classifier.bias          | MISSING    |
classifier.weight        | MISSING    |
pre_classifier.weight    | MISSING    |

Notes:
- UNEXPECTED    :can be ignored when loading from different
task/architecture; not ok if you expect identical arch.
- MISSING       :those params were newly initialized because missing from the
checkpoint. Consider training on your downstream task.
/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.py:775:
UserWarning: 'pin_memory' argument is set as true but no accelerator is
found, then device pinned memory won't be used.
  super().__init__(loader)
```

[742/1250 2:19:19 < 1:35:38, 0.09 it/s, Epoch 1.19/2]

Epoch	Training Loss	Validation Loss	Accuracy	F1
1	0.634274	0.571827	0.800000	0.795082

```
/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.py:775:
UserWarning: 'pin_memory' argument is set as true but no accelerator is
found, then device pinned memory won't be used.
  super().__init__(loader)
/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.py:775:
UserWarning: 'pin_memory' argument is set as true but no accelerator is
found, then device pinned memory won't be used.
  super().__init__(loader)
```

[760/1250 2:22:21 < 1:32:01, 0.09 it/s, Epoch 1.21/2]

Epoch	Training Loss	Validation Loss	Accuracy	F1
1	0.634274	0.571827	0.800000	0.795082